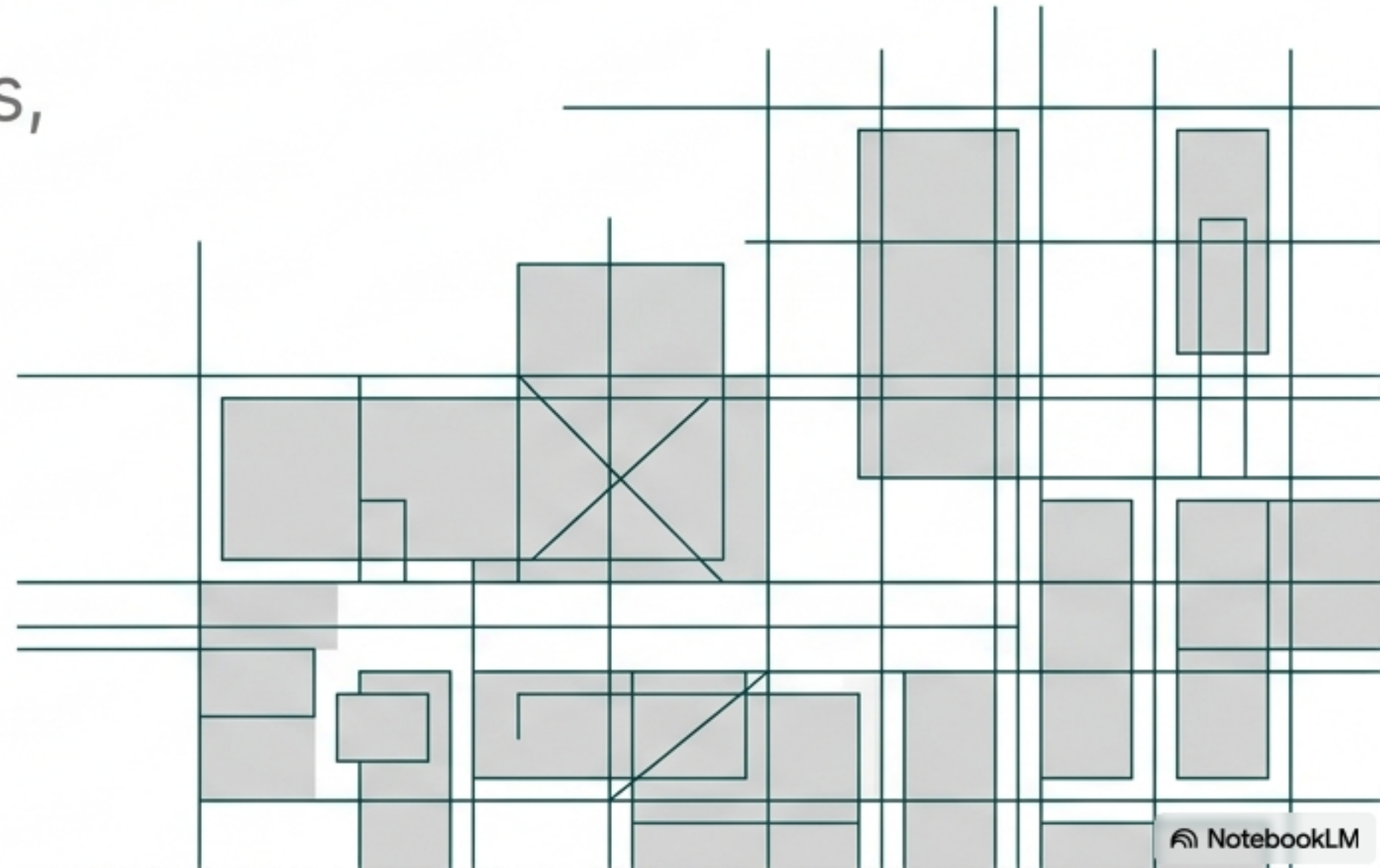


The Fundamentals of Software Architecture

Balancing Trade-offs, Frameworks,
and Proven Blueprints.



Architecture is a Living Foundation

Architectural models serve two distinct mandates across the engineering lifecycle.



High-Level Discussion

- Facilitates stakeholder alignment and requirement negotiation.
- Abstract focus prevents getting bogged down in technical noise.

Implementation Blueprint

- Provides rigorous, unambiguous documentation for developers.
- Prevents synchronization debt where code and design dangerously diverge.

The Art of Compromise

*You cannot optimize for everything. Component **grain-size** dictates system behavior.*



Performance

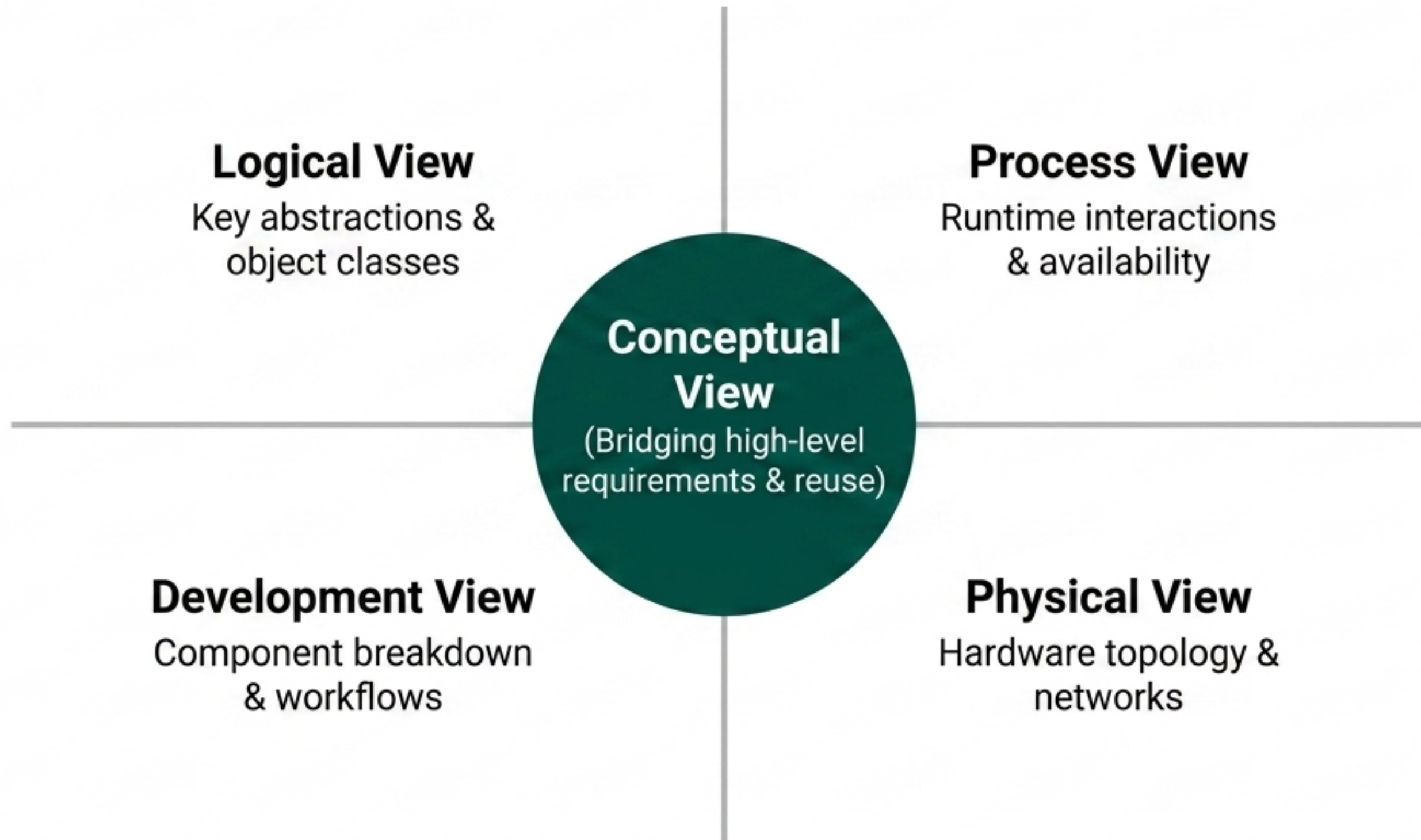
- Architecture relies on large, monolithic components.
- **Benefit:** Minimizes inter-component communication overhead and latency.

Maintainability

- Architecture relies on small, fine-grain, self-contained components.
- **Benefit:** Localizes changes, separates data producers/consumers, and prevents ripple effects.

The Multi-Perspective Approach

No single diagram can capture a complex system. A multi-view framework is required.



Mapping Views to Stakeholders

Tailor the documentation entirely to the professional relying on it.



Logical View

Audience: Designers & Business Analysts

Focus: Maps functional requirements directly to operational objects and classes.



Process View

Audience: System Architects & Integrators

Focus: Evaluates non-functional metrics: runtime performance, concurrency, and system availability.

Organizing Work & Hardware

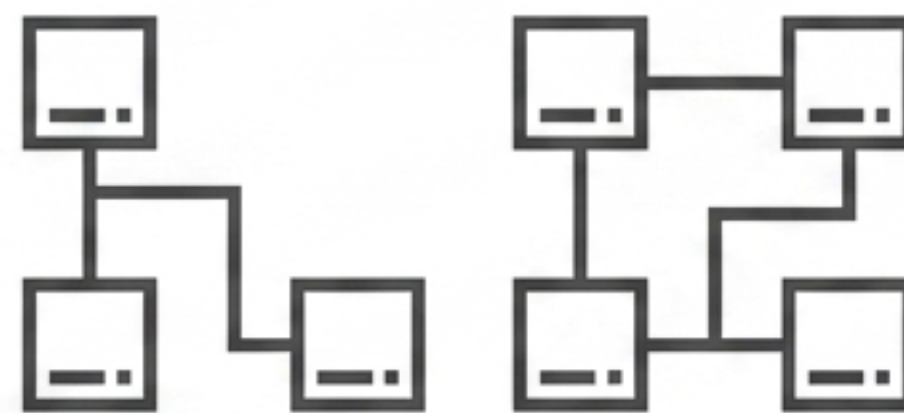
Translating abstract logic into tangible workflows and deployment topology.



Development View

Audience: Software Managers & Programmers

Focus: Breaking software down into manageable work packages and discrete team assignments.



Physical View

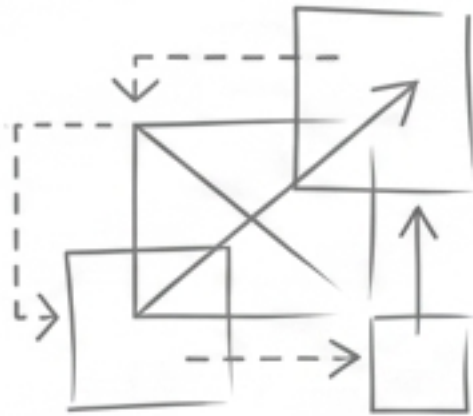
Audience: Systems Engineers

Focus: Hardware environment details, network configuration, and physical software distribution.

The Velocity vs. Precision Trade-off

Documentation that is not actively used for communication is waste.

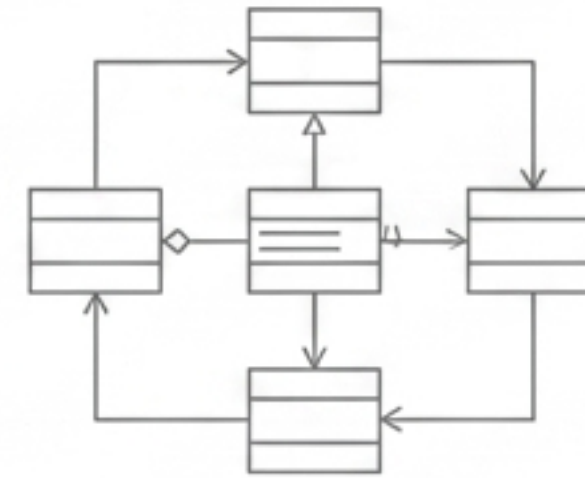
Informal Notations (Whiteboarding)



Best for: Active design, agile environments, implementation-agnostic planning.

Advantage: Maintains design velocity; perfectly matches the abstraction level needed early on.

Formal UML (Unified Modeling Language)

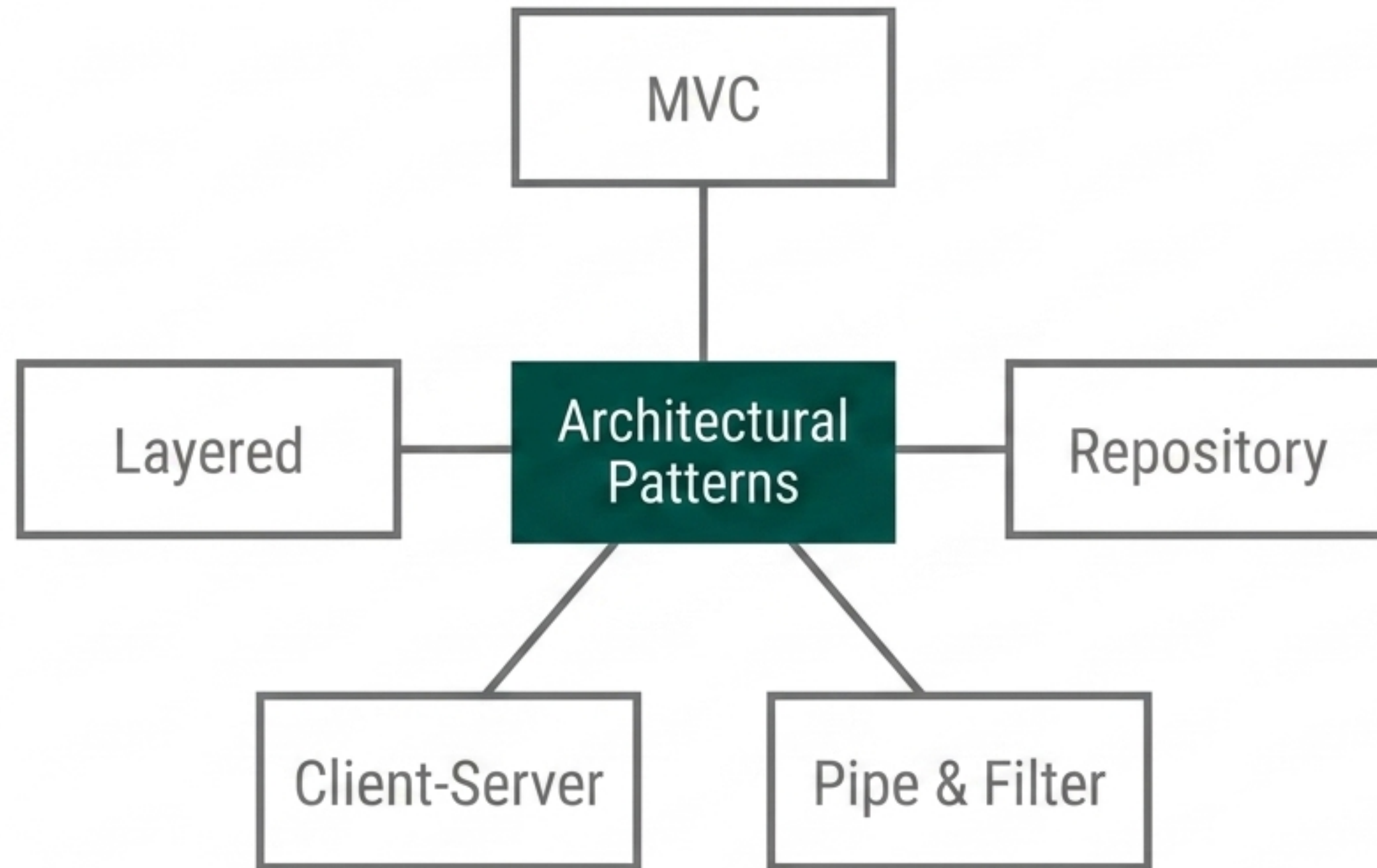


Best for: Critical systems, long-term high-stakes maintenance, model-driven development.

Advantage: Provides a rigorous, detailed, unambiguous blueprint for developers.

The Proven Blueprints

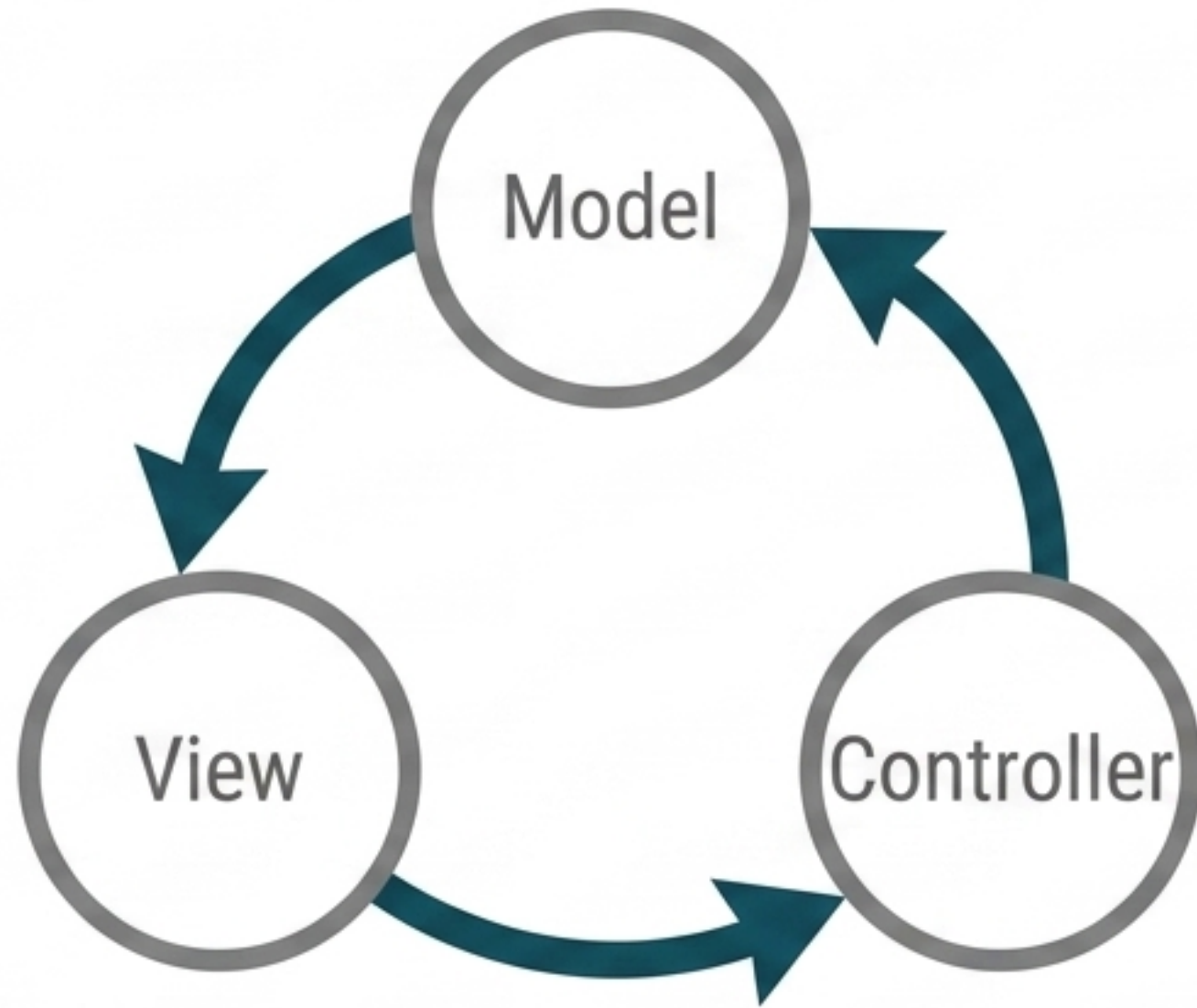
Do not start from scratch. Use stylized, abstract descriptions of tested good practice.



- Patterns provide a roadmap for organizing data and processing.
- Each highlights inherent system trade-offs before code is written.
- The Goal: Prevent Architectural Drift—the slow degradation of design integrity when structures are mixed without discipline.

MVC: Separating Presentation

Decouples internal data logic from user interaction.



1. **Model:** Manages system data. Notifies View of state changes.
2. **View:** Renders data. Requests updates from Model.
3. **Controller:** Manages user events. Maps actions to Model updates.

Best use: When identical data must be represented in multiple formats simultaneously.

Trade-off: Introduces unjustifiable code complexity for simple, static data models.

Layered: Organizing by Responsibility

Isolates machine-dependent code to localize changes and increase portability.

L4: User Interface

L3: UI Management (Auth, Identity)

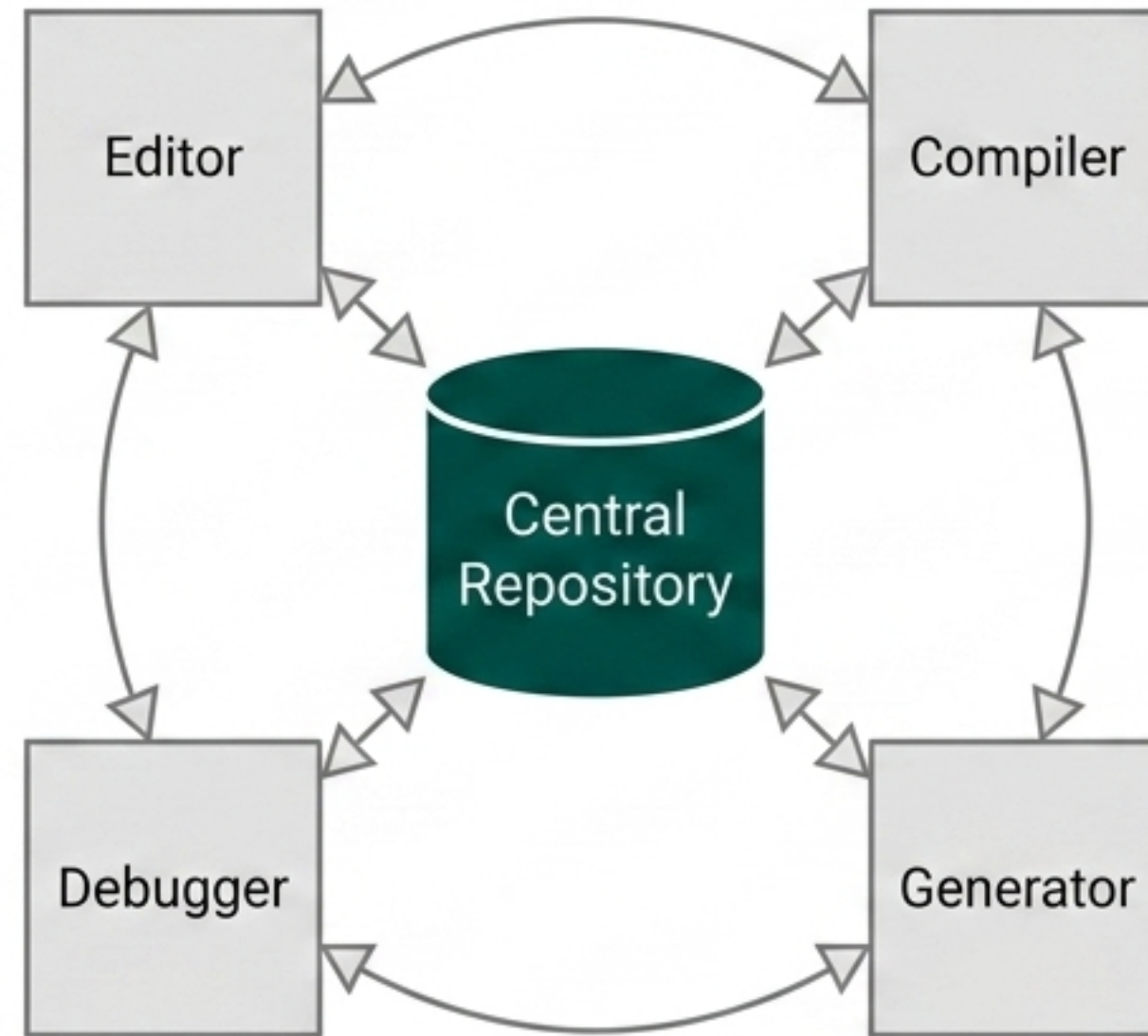
L2: Application Core (Business Logic)

L1: System Support (OS, Database)

- Each layer relies only on the services of the layer immediately below it.
- Example: The iLearn system ensures portable access control via a dedicated Layer 3.
- Trade-off: Performance drops due to multiple levels of request interpretation traversing the stack.

Repository: Centralized Data Sharing

Ideal for data-driven systems requiring absolute consistency.



- Components never communicate directly; they interact exclusively via the central hub.
- Highly efficient for sharing massive volumes of data.
- Trade-off: The repository creates a critical bottleneck and a catastrophic single point of failure.

Client-Server: Distributed Services

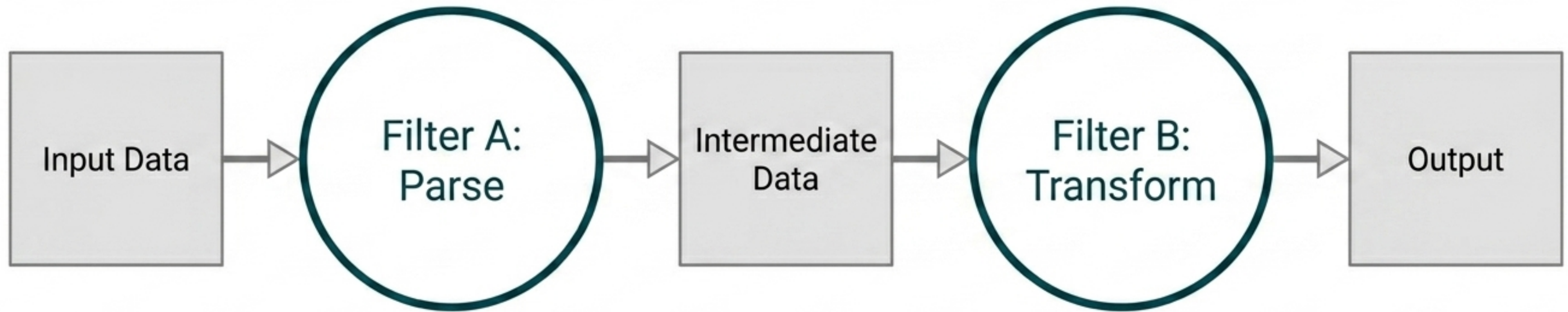
Offers exceptional scalability across distributed geographical locations.



- Separates specialized service providers from end-user consumers.
- Servers can be transparently upgraded, replaced, or replicated.
- Trade-off: Performance is entirely network-dependent; uniquely susceptible to Denial-of-Service (DoS) attacks.

Pipe & Filter: Data Transformation

Streamlines processing through a sequence of discrete, independent functional transformations.



- Perfectly matches real-world batch sequential workflows (e.g., Invoice processing).
- Highly evolvable; individual filters can be swapped without disrupting the pipeline.
- Trade-off: Massive system overhead caused by continuous parsing and unparsing to agreed data formats.

Selecting the Right Tool

Most robust systems are hybrids; apply patterns to match critical priorities.

Architectural Pattern	Primary Advantage	Primary Disadvantage
MVC	Decouples data from UI	Unjustifiable code complexity for simple apps
Layered	High portability & localized change	Slower performance from traversal
Repository	Absolute data consistency	Critical single point of failure
Client-Server	Highly scalable & distributed	Network dependent / DoS risk
Pipe & Filter	Easy to evolve business workflows	High parsing/unparsing overhead

The Architect's Directive

Consistency and utility must always supersede exhaustive, unused detail.

1

Prioritize Utility

Cease documentation the moment it successfully facilitates communication or acts as a sufficient blueprint.

2

Localize Change

Favor patterns that keep components independent. A UI change should never force a database rewrite.

3

Acknowledge Trade-offs

Pure optimization is a myth. Success lies in intentionally balancing performance against long-term maintainability.